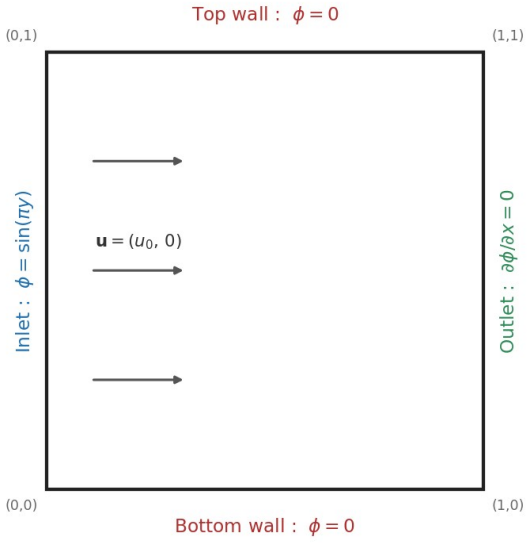
A Cell-Centred Finite Volume Solver for the 2D Convection–Diffusion Equation on Unstructured Grids

Development and Validation

Unit-square domain and boundary conditions



A hands-on computational study for heat and mass transfer analysis

1. Introduction

1.1 The convection–diffusion problem

Almost every heat- and mass-transfer problem comes down to answering one question: how does some quantity — temperature, a chemical concentration, a pollutant — spread through a moving medium? Two physical effects decide the answer. The first is convection: the quantity is simply carried along by the bulk motion of the fluid, the way a dye is swept downstream by a river. The second is diffusion: the quantity spreads on its own from regions of high value to low, the way that same dye slowly blurs outward even in still water. Real problems always involve both at once, and the interesting behaviour lives in how they compete.

That competition is measured by a single dimensionless number, the **Péclet number**, defined as Pe = uL/α, where u is the flow speed, L a length scale and α the diffusion coefficient. When Pe is small, diffusion wins and the field is smooth. When Pe is large, convection wins, gradients steepen and thin boundary layers appear. A solver that behaves well across this whole range is exactly what is needed for practical heat- and mass-transfer analysis, and building one is the purpose of this work.

1.2 Governing equation and test problem

The steady, two-dimensional scalar convection–diffusion equation solved here is

$$\partial(u\varphi)/\partial x + \partial(v\varphi)/\partial y \;=\; \alpha\,(\,\partial^2\varphi/\partial x^2 + \partial^2\varphi/\partial y^2\,),$$

on the unit square, with a purely horizontal velocity (u, v) = (u_0, 0). The boundary conditions, shown on the domain sketch in Section 3, are a fixed inlet profile φ = sin(πy) on the left, zero value on the top and bottom walls, and a zero-gradient (∂φ/∂x = 0) outflow on the right. This problem is chosen deliberately because it has a known **analytic steady solution**, which lets every numerical result be checked against an exact answer rather than merely looking plausible.

For these boundary conditions the problem admits a closed-form steady solution, given in the reference problem statement [1] as

$$\varphi^{exact}(x,y) \;=\; \sin(\pi y)\cdot\left[\,r_2\,e^{(r_1x + r_2L)} - r_1\,e^{(r_1L + r_2x)}\,\right] / \left[\,r_2\,e^{(r_2L)} - r_1\,e^{(r_1L)}\,\right],$$

$$\text{where}\quad r_{1,2} \;=\; u_0/2\alpha \;\pm\; \sqrt{(\,u_0^2/4\alpha^2 \;+\; \pi^2\,)}.$$

This analytic field is what every numerical result in this report is measured against. All results below use u = 1 and L = 1, so that the Péclet number is set directly by the diffusion coefficient (α = 1/Pe). The main validation case is run at Pe = 100 — a strongly convection-dominated regime that stresses the scheme — and the influence of the Péclet number itself is examined separately at Pe = 1, 10 and 100.

1.3 Objective

The aim of this study is to develop a finite-volume solver for the above problem on unstructured triangular meshes, and then to demonstrate, with evidence, that it works: that its solution matches the exact field, that its error falls at the expected rate as the mesh is refined, that it captures the correct physics as the Péclet number changes, and that both an explicit and an implicit time-marching path reach the same answer with very different efficiency.

2. Solver Development and Features

The solver is a cell-centred finite-volume code: one unknown value of φ is stored at the centroid of each triangular cell, and the governing equation is enforced by balancing the fluxes that cross the cell's faces.

Rather than looping over cells, the code loops over faces — the same face-based organisation used by production codes such as OpenFOAM and SU2 — so that each face computes its flux once and hands equal and opposite contributions to the two cells it separates. This makes conservation exact by construction: whatever leaves one cell enters its neighbour.

The code is written in C++ using an object-oriented structure. The mesh, the field, the convection scheme and the time integrator are each their own component, which keeps the numerics readable and makes it easy to switch a scheme on or off. The mesh geometry and connectivity are read from external files, and the run is controlled by a small plain-text input file, so no recompilation is needed to change a case. Results are written in both Tecplot and VTK formats, together with the full residual history. The principal features are:

- **Convection**: first-order upwind, or second-order upwind with a gradient-based reconstruction.
- **Diffusion**: a Green–Gauss gradient built over the face "diamond" (the co-volume formed by the two cell centroids and the two face nodes), which is what a distorted triangular mesh needs to remain accurate.
- **Nodal interpolation**: cell values are interpolated to the mesh nodes by either an area-weighted average or a linearity-preserving pseudo-Laplacian, with a dedicated least-squares treatment at the outlet.
- **Time marching**: an explicit multi-stage Runge–Kutta scheme and an implicit symmetric Gauss–Seidel scheme, with local or global time-stepping and CFL ramping for the implicit path.
- **Validation**: every run is compared against the analytic solution through a global, area-weighted L2 error norm.

The numerical methodology follows the finite-volume framework described by Moukalled, Mangani and Darwish [2], and the specific viscous (diffusion) discretisation on unstructured grids follows the co-volume procedure developed in the doctoral thesis of Munikrishna [1].

3. Test Case and Computational Domain

3.1 Domain and boundary conditions

The computational domain is a unit square. The boundary conditions are summarised in Figure 1: a prescribed sinusoidal inlet on the left edge, no-value walls on the top and bottom, and a zero-gradient outflow on the right edge. The flow is uniform and horizontal, carrying the scalar from the inlet towards the outlet while diffusion spreads it vertically.

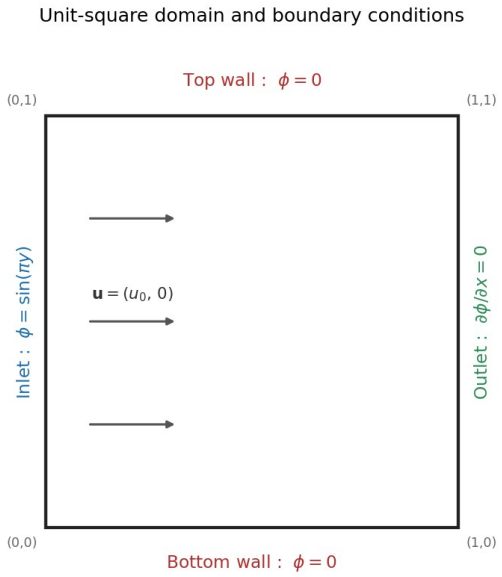


Figure 1. *Unit-square domain, uniform horizontal velocity, and the four boundary conditions.*

3.2 Computational meshes

The domain is discretised with unstructured triangular meshes that are intentionally distorted and stretched, so that they resemble the kind of high-aspect-ratio grids used to resolve boundary layers. Four successively refined meshes are used, of 106, 424, 1696 and 6784 cells (68, 241, 905 and 3505 points), each refinement roughly quadrupling the cell count. Figure 2 shows the coarsest and the finest of these.

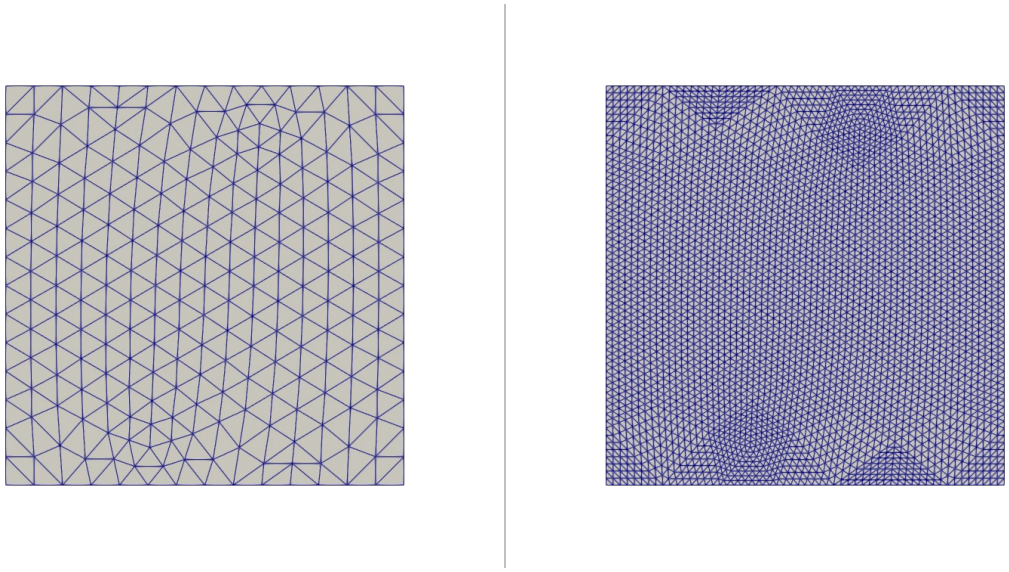


Figure 2. *Triangular meshes: coarsest, 106 cells (left) and finest, 6784 cells (right).*

4. Solver Settings

Table 1 lists the settings used for the main validation and grid-convergence study (Pe = 100). The same spatial discretisation is then used for two additional comparisons: the explicit-versus-implicit study (Table

2) and the Péclet-number study, in which only the diffusion coefficient is changed to give Pe = 1, 10 and 100.

Table 1. Baseline solver settings (Pe = 100 case).

Parameter	Value
Péclet number, Pe	100
Convection scheme	Second-order upwind
Diffusion	Green–Gauss (diamond co-volume)
Nodal interpolation	Pseudo-Laplacian
Time integration	Implicit symmetric Gauss–Seidel
Time step	Global
Convergence tolerance	1×10^{-10}
Meshes (grid study)	106, 424, 1696, 6784 cells

Table 2. Time-integration settings for the explicit-versus-implicit comparison.

Setting	Explicit	Implicit
Method	3-stage Runge–Kutta	Symmetric Gauss–Seidel
CFL number	0.2 (constant)	ramped
SGS sweeps	—	14
Tolerance	1×10^{-10}	1×10^{-10}
Iterations to converge	190	23

In both cases the spatial scheme is identical (second-order upwind, pseudo-Laplacian nodes, global time step), so the two runs must reach the same steady state and differ only in how quickly they get there.

5. Results and Discussion

5.1 Solution field and validation

Figures 3 and 4 compare the computed field (right panel) against the exact solution (left panel) on the coarse and fine meshes. On both meshes the numerical solution reproduces the correct structure: a sinusoidal profile in the vertical direction that is carried across the domain and gently thinned towards the outlet. On the coarse mesh the contours are visibly faceted, simply because there are few cells to represent the field; on the fine mesh the computed field and the exact field are almost indistinguishable.

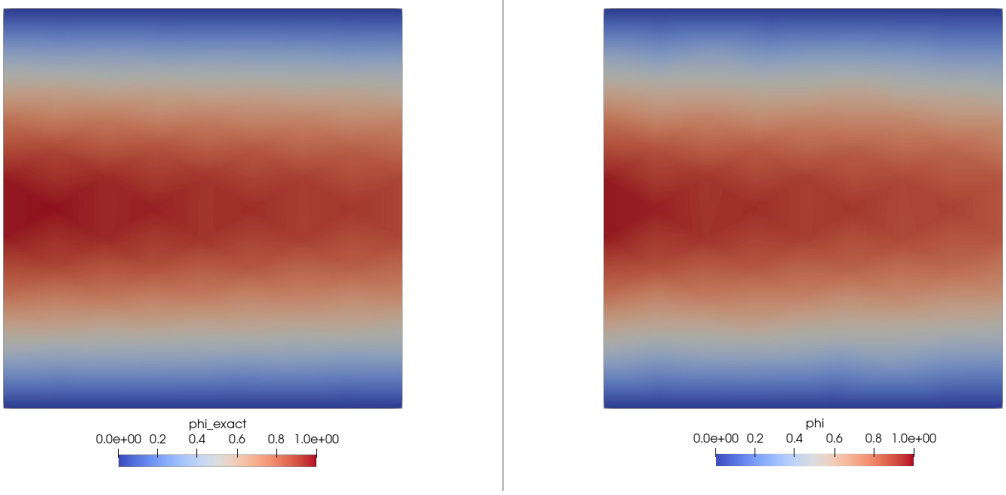


Figure 3. *Coarse mesh (106 cells), Pe = 100: exact φ (left) and computed φ (right).*

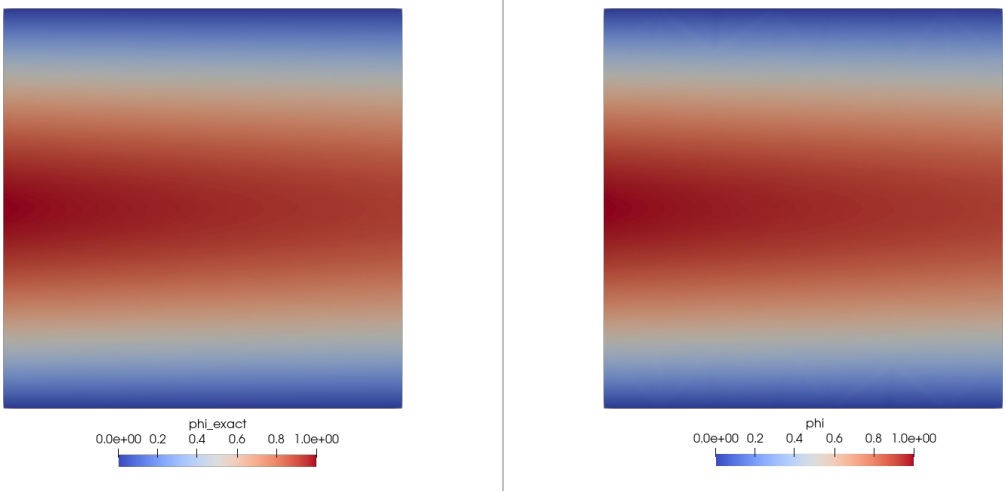


Figure 4. *Fine mesh (6784 cells), Pe = 100: exact φ (left) and computed φ (right). The two are visually identical.*

5.2 Grid convergence and order of accuracy

To measure accuracy properly, the global L2 error against the exact solution was recorded on all four meshes (Table 3). The error falls steadily as the mesh is refined, dropping by very nearly a factor of three each time the cell count quadruples.

Table 3. Global L2 error versus mesh size (Pe = 100).

Points (np)	Cells (nc)	L2 error
68	106	4.902×10^{-3}
241	424	1.668×10^{-3}
905	1696	6.049×10^{-4}
3505	6784	2.026×10^{-4}

Plotting these errors against the mesh spacing on logarithmic axes (Figure 5) gives a straight line whose slope is the **order of accuracy**. A least-squares fit gives a slope of about 1.53. This sits between first and second order, which is exactly what is expected for this method: the diffusion discretisation on distorted triangles is, as noted in Munikrishna's thesis [1], locally inconsistent yet globally convergent, and the observed rate of roughly one-and-a-half is the well-documented result of that behaviour. Expressed against the thesis's own measure of grid size (dx = 1/np), the same data give a fall rate close to the value reported there, confirming that the solver reproduces the reference behaviour.

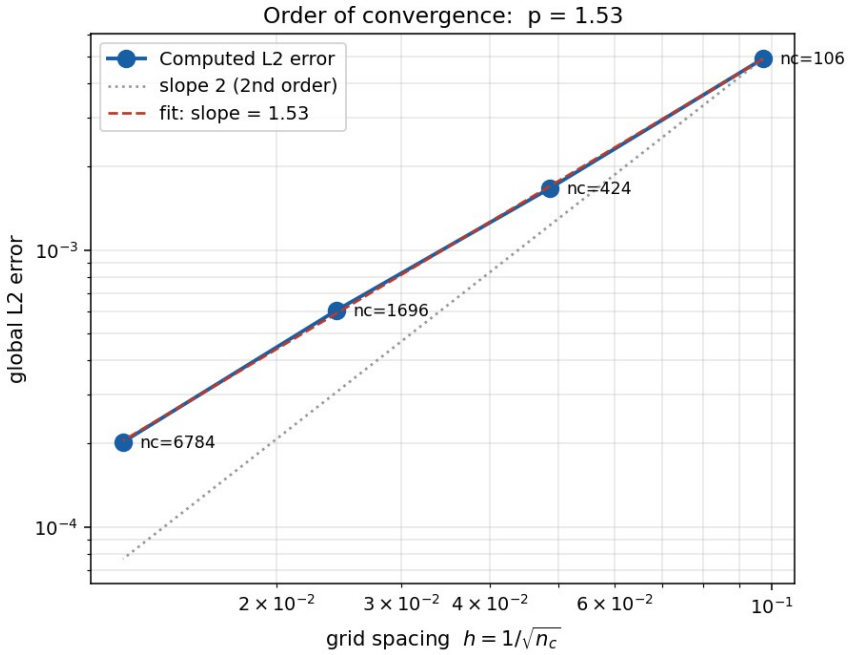


Figure 5. *Grid convergence: global L2 error versus mesh spacing h = 1/√nc. The fitted slope is 1.53; a slope-2 line is shown for reference.*

5.3 Effect of the Péclet number

Figure 6 shows equally-spaced contours of φ on the fine mesh for Pe = 1, 10 and 100. The change in physics is clear. At Pe = 1 (left) diffusion is strong: the contours are rounded and symmetric, and the scalar spreads freely in all directions. As the Péclet number rises, convection increasingly dominates: the contours are stretched in the flow direction, the field is swept towards the outlet, and by Pe = 100 (right) the contours are almost horizontal through the interior with the variation squeezed into a thin layer. This is precisely the convection-dominated behaviour the solver was built to handle, and it captures the transition cleanly and without spurious oscillations.

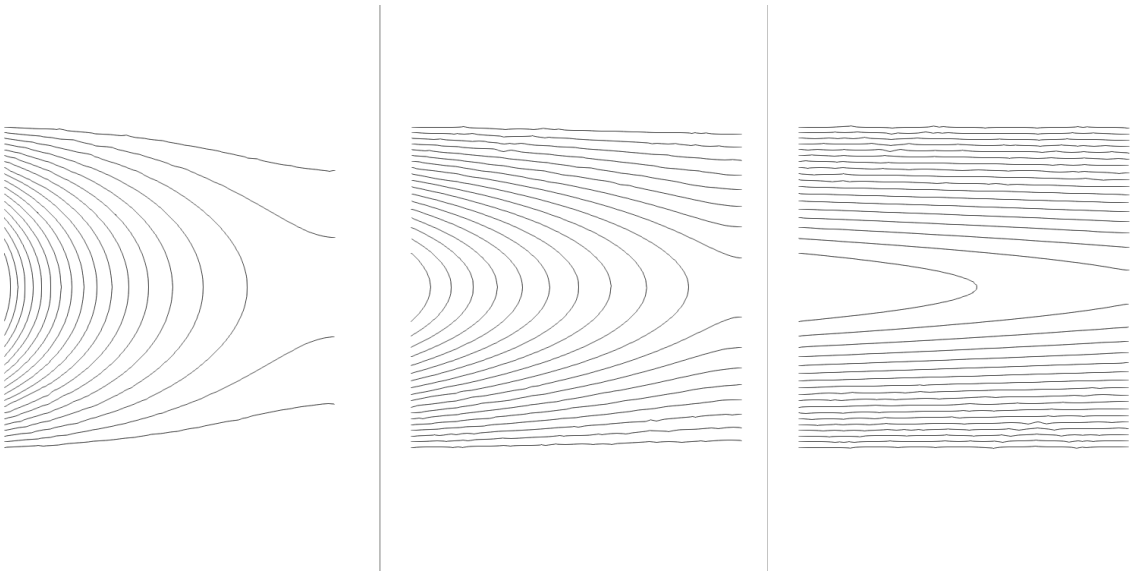


Figure 6. *Iso-contours of φ on the fine mesh: Pe = 1 (left), Pe = 10 (middle), Pe = 100 (right). Higher Pe stretches the contours in the flow direction.*

5.4 Explicit versus implicit marching

The same case was run with both the explicit and the implicit time-marching schemes. Figure 7 shows how the normalised residual falls with iteration. Both schemes drive the residual monotonically down to the 10^{-10} tolerance, but they do so very differently: the explicit scheme needs 190 iterations and spends a long time on a plateau, because its stable time step is capped, whereas the implicit scheme, free of that cap and using a ramped CFL number, reaches the same tolerance in just 23 iterations — roughly an eight-fold reduction.

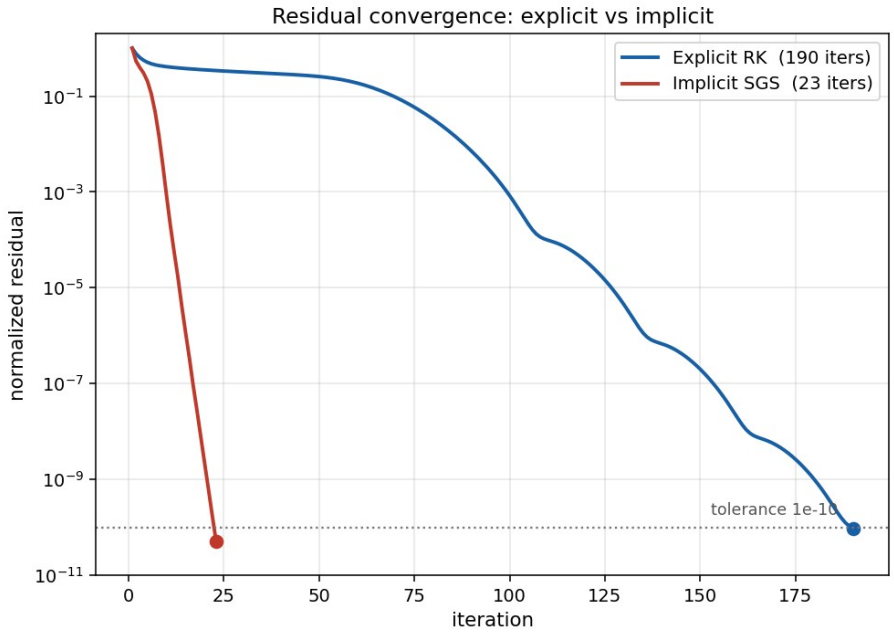


Figure 7. *Residual convergence: explicit RK (190 iterations) versus implicit SGS (23 iterations) to the same 10^{-10} tolerance.*

Crucially, the faster convergence costs nothing in accuracy. Figure 8 plots φ along a horizontal line through the domain for the exact solution and for both schemes. The explicit and implicit curves lie exactly on top of one another — confirming that they reach the identical steady state — and both track the exact solution, with the small, uniform offset being the discretisation error of that particular mesh, not a difference between the two methods. In short, the implicit scheme is the better choice: the same answer, reached far more cheaply.

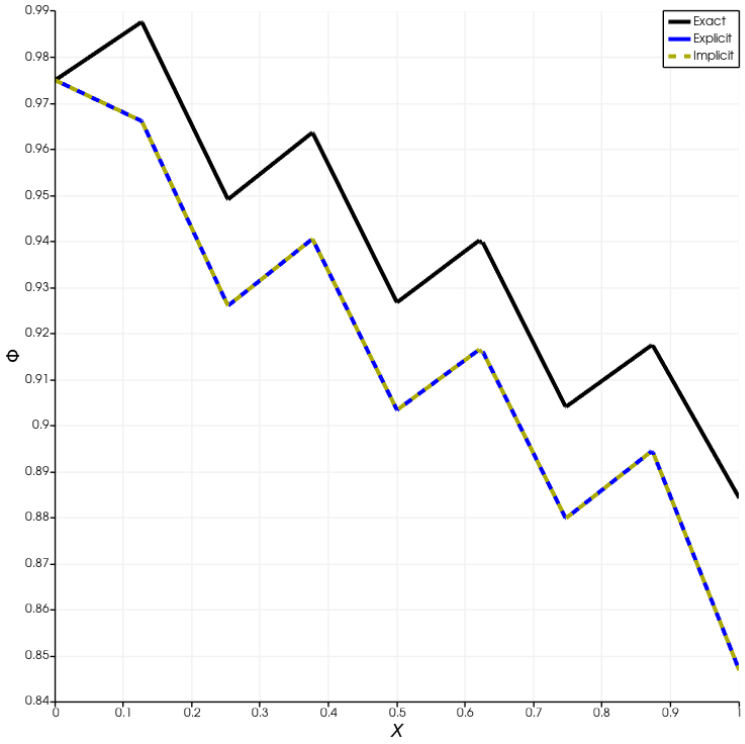


Figure 8. *φ along a line through the domain: exact (black) versus explicit (blue) and implicit (dashed). The explicit and implicit results coincide.*

6. Conclusion

An unstructured, cell-centred finite-volume solver for the two-dimensional convection–diffusion equation has been developed and thoroughly validated. Across every test the solver behaves as it should. Its computed field matches the analytic solution, and the match sharpens as the mesh is refined; the global error falls at an order of about 1.53, consistent with the reference method. As the Péclet number is increased from 1 to 100 the solution transitions cleanly from diffusion-dominated to convection-dominated behaviour, with no spurious oscillations in this demanding regime. Finally, the implicit symmetric Gauss–Seidel scheme reaches the same steady state as the explicit scheme in roughly one-eighth of the iterations, making it the practical choice for this class of problem.

Taken together, these results demonstrate a working, verified computational tool and, with it, a solid hands-on grounding in the finite-volume methods that underpin computational heat- and mass-transfer analysis. The modular structure of the code leaves clear room for extension — additional convection schemes, other boundary conditions, or coupling the scalar transport to a computed flow field — as natural next steps.

References

[1] N. Munikrishna, *On Viscous Flux Discretization Procedures for Finite Volume and Meshless Solvers*, Ph.D. Thesis, Department of Aerospace Engineering, Indian Institute of Science, Bengaluru.

[2] F. Moukalled, L. Mangani and M. Darwish, *The Finite Volume Method in Computational Fluid Dynamics: An Advanced Introduction with OpenFOAM and Matlab*, Springer, 2016.